

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

TESTOVANIE MUTÁCIÍ S PODPOROU UMELEJ
INTELIGENCIE: INTELIGENTNÉ PROCESY PRE
AUTOMATIZOVANÉ ZABEZPEČENIE KVALITY
DIPLOMOVÁ PRÁCA

2026

BC. ONDREJ BABINSKÝ

UNIVERZITA KOMENSKÉHO V BRATISLAVE
FAKULTA MATEMATIKY, FYZIKY A INFORMATIKY

TESTOVANIE MUTÁCIÍ S PODPOROU UMELEJ
INTELIGENCIE: INTELIGENTNÉ PROCESY PRE
AUTOMATIZOVANÉ ZABEZPEČENIE KVALITY
DIPLOMOVÁ PRÁCA

Študijný program: Aplikovaná informatika
Študijný odbor: Informatika
Školiace pracovisko: Katedra informatiky
Školiteľ: doc. Ing. Ivan Polášek, PhD.
Konzultant: Martin Marko

Bratislava, 2026
Bc. Ondrej Babinský



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Ondrej Babinský
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Testovanie mutácií s podporou umelej inteligencie: Inteligentné procesy pre automatizované zabezpečenie kvality
AI-Assisted Mutation Testing: Intelligent Pipelines for Automated Quality Assurance

Anotácia: Tradičné testovanie mutácií čelí kritickým výzvam – konkrétne neefektívnosti náhodných mutácií, vysokej spotrebe zdrojov a ťažkostiam pri interpretácii výsledkov –, ktoré bránia jeho širokému uplatneniu. Tento projekt skúma a navrhuje metódu a nástroj, ktorý demonštruje, ako umelá inteligencia môže optimalizovať tento pracovný postup. Hlavným zameraním je určiť, ako môže umelá inteligencia pomôcť pri definovaní inteligentných cieľov mutácií, výbere relevantných mutátorov, vytváraní špecializovaných sémantických mutácií a automatizácii interpretácie správ, aby sa usmernili následné testy alebo opravy kódu.

Cieľ: Cieľom je implementovať prototyp, ktorý dokáže, že umelá inteligencia je prospešným doplnkom testovania mutácií, a to prostredníctvom koordinácie existujúcich rámcov (napr. Stryker, Cosmic Ray) s vlastnou logikou umelej inteligencie. Riešenie bude spracovávať celý proces – od inteligentného vykonávania a hľadania klasifikácie až po automatické opravy – a bude navrhnuté tak, aby sa bezproblémovo integrovalo s modernými asistentmi kódovania (napr. Roo Code, Gemini CLI). Cieľom práce bude implementácia modulárneho procesu, ktorý umožní buď širokú implementáciu od začiatku do konca, alebo hlboké technické zameranie na konkrétne moduly s vysokou hodnotou.

Vedúci: doc. Ing. Ivan Polášek, PhD.
Konzultant: Martin Marko
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.
Dátum zadania: 28.11.2025

Dátum schválenia: 04.12.2025
prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

Pod'akovanie: Ďakujem svojmu konzultantovi Martinovi Markovi za výraznú technickú pomoc, cenné rady a priebežné konzultácie pri návrhu aj riešení práce. Zároveň ďakujem školiteľovi doc. Ing. Ivanovi Poláškov, PhD., za odborné vedenie a pripomienky počas práce.

Abstrakt

Testovanie mutácií je technika primárne na hodnotenie kvality testov, pri ktorej sa do programu zámerne vkladajú malé chyby a sleduje sa, či ich testy dokážu odhaliť. Napriek svojmu potenciálu sa v praxi nepoužíva tak často, pretože môže byť časovo náročné, generovať veľa málo užitočných mutácií a jeho výsledky nemusia byť pre vývojára hneď jasné. Táto práca sa zaoberá tým, ako možno tento proces zjednodušiť pomocou veľkých jazykových modelov a agentových nástrojov. Praktická časť sa zameriava najmä na prácu s nástrojom Cosmic Ray, obmedzenie mutácií na vybrané časti kódu a návrh pracovného postupu, v ktorom vie AI asistent pomôcť s prípravou konfigurácie, spustením testovania a ďalšími krokmi. Cieľom je vytvoriť prototyp, ktorý ukáže, či môže byť testovanie mutácií jednoduchšie a použiteľnejšie v bežnom vývojárskom procese.

Kľúčové slová: testovanie mutácií, Cosmic Ray, MCP, zabezpečenie kvality softvéru

Abstract

Mutation testing is a technique for evaluating the quality of tests by intentionally inserting small faults into a program and checking whether the existing tests can detect them. Despite its potential, it is not used very often in practice, because it can be time-consuming, produce many less useful mutations, and its results are not always immediately clear to developers. This thesis focuses on how this process can be simplified using large language models and agent-based tools. The practical part mainly works with the Cosmic Ray tool, limiting mutations to selected parts of the code and designing a workflow where an AI assistant can help with preparing the configuration, running mutation testing, and supporting the next steps. The goal is to create a prototype showing whether mutation testing can become simpler and more usable in a regular software development workflow.

Keywords: mutation testing, Cosmic Ray, MCP, software quality assurance

Obsah

Úvod	1
1 Testovanie mutácií	2
1.1 Základný princíp testovania mutácií	2
1.2 Mutanti, mutačné operátory a mutačné skóre	2
1.3 Zabité, preživšie a ekvivalentné mutanty	2
1.4 Porovnanie s pokrytím kódu	2
1.5 Praktické problémy testovania mutácií	2
2 Existujúce nástroje a prístupy	3
2.1 Prehľad nástrojov pre testovanie mutácií v jazyku Python	3
2.2 Porovnanie vybraných nástrojov	3
2.3 Nástroj Cosmic Ray	3
2.4 Testovanie mutácií v praxi	3
2.5 Využitie veľkých jazykových modelov pri testovaní mutácií	3
3 Návrh AI asistovaného procesu testovania mutácií	4
3.1 Motivácia pre cielené testovanie mutácií	4
3.2 Obmedzenie mutácií na vybrané časti kódu	4
3.3 Výber relevantných testov	4
3.4 Úloha AI asistenta v navrhovanom procese	4
3.5 Návrh pracovného postupu	4
4 Implementácia prototypu	5
4.1 Rozšírenie nástroja Cosmic Ray o filtrovanie podľa riadkov	5
4.2 Úprava HTML správy pre skrytie preskočených mutantov	5
4.3 Mutation MCP Server	5
4.4 Integrácia s AI asistentom	5
4.5 Štruktúra konfigurácie a spúšťania experimentov	5
5 Experimentálne overenie	6
5.1 Výber testovaných projektov	6

5.2	Experiment s knižnicou validators	6
5.3	Porovnanie základného a cieleného spustenia	6
5.4	Vyhodnotenie času behu a počtu mutantov	6
5.5	Analýza preživších mutantov	6
6	Diskusia	7
6.1	Prínosy navrhnutého riešenia	7
6.2	Obmedzenia prototypu	7
6.3	Možnosti využitia veľkých jazykových modelov	7
6.4	Hrozby pre platnosť vyhodnotenia	7
7	Záver	8
7.1	Zhrnutie výsledkov	8
7.2	Splnenie cieľov práce	8
7.3	Možnosti ďalšieho rozšírenia	8

Zoznam obrázkov

Zoznam tabuliek

Úvod

Testovanie softvéru je jednou zo základných častí vývoja, pretože pomáha odhaľovať chyby ešte predtým, ako sa dostanú k používateľom. Samotná existencia testov však automaticky neznamená, že testy sú kvalitné. Program môže mať vysoké pokrytie riadkov alebo vetiev, no stále môže obsahovať chyby, ktoré testy nedokážu zachytiť. Jednou z techník, ktorá sa snaží tento problém riešiť, je mutation-testing.

Mutation-testing funguje tak, že do programu zámerne vkladá malé zmeny, ktoré predstavujú možné chyby. Takto upravené verzie programu sa nazývajú mutanti. Ak existujúce testy dokážu zmenu odhaliť, mutant je považovaný za zabitého. Ak testy prejdú aj napriek vlozenej chybe, môže to naznačovať, že testovacia sada nie je dostatočne silná. Vďaka tomu je mutation testing užitočný nástroj na hodnotenie kvality testov a hľadanie miest, kde je potrebné testy doplniť alebo zlepšiť.

Napriek svojmu potenciálu sa táto technika v bežnej praxi nepoužíva tak často, ako by mohla. Hlavným dôvodom je jej časová náročnosť. Tradičné nástroje môžu vytvoriť veľké množstvo mutantov, ktorých vyhodnocovanie zaberá veľa času a výpočtových zdrojov. Zároveň nie všetky mutácie sú pre vývojára rovnako užitočné. Niektoré sa týkajú nepodstatných častí kódu, iné sú ťažko interpretovateľné alebo neprinášajú jasnú informáciu o tom, ako by sa mali testy zlepšiť. To znižuje praktickú hodnotu mutation testingu v každodennom vývojárskom procese.

Cieľom tejto práce je preskúmať, ako možno testovanie mutácií zefektívniť pomocou veľkých jazykových modelov a inteligentných automatizovaných procesov. Práca sa zameriava napríklad na scenár, v ktorom vývojár nechce spúšťať mutation testing nad celým projektom, ale iba nad vybranými časťami kódu, napríklad nad riadkami, ktoré boli zmenené alebo refaktorované. Praktická časť tu pracuje s nástrojom Cosmic Ray a rieši rozšírenie jeho použitia tak, aby bolo možné obmedziť mutácie na nakonfigurované rozsahy riadkov.

Výsledkom práce má byť prototyp modulárneho procesu, ktorý ukáže, či môže kombinácia existujúceho mutation testing nástroja, vlastnej filtrovacej logiky a AI asistenta zvýšiť praktickú použiteľnosť tejto techniky.

Kapitola 1

Testovanie mutácií

1.1 Základný princíp testovania mutácií

Mutation-Guided LLM-based Test Generation at Meta [1]

Mutation Testing in Practice: Insights From Open-Source Software Developers [2]

1.2 Mutanti, mutačné operátory a mutačné skóre

1.3 Zabité, preživšie a ekvivalentné mutanty

1.4 Porovnanie s pokrytím kódu

1.5 Praktické problémy testovania mutácií

Kapitola 2

Existujúce nástroje a prístupy

- 2.1 Prehľad nástrojov pre testovanie mutácií v jazyku Python
- 2.2 Porovnanie vybraných nástrojov
- 2.3 Nástroj Cosmic Ray
- 2.4 Testovanie mutácií v praxi
- 2.5 Využitie veľkých jazykových modelov pri testovaní mutácií

Kapitola 3

Návrh AI asistovaného procesu testovania mutácií

- 3.1 Motivácia pre cielené testovanie mutácií
- 3.2 Obmedzenie mutácií na vybrané časti kódu
- 3.3 Výber relevantných testov
- 3.4 Úloha AI asistenta v navrhovanom procese
- 3.5 Návrh pracovného postupu

Kapitola 4

Implementácia prototypu

- 4.1 Rozšírenie nástroja Cosmic Ray o filtrovanie podľa riadkov
- 4.2 Úprava HTML správy pre skrytie preskočených mutantov
- 4.3 Mutation MCP Server
- 4.4 Integrácia s AI asistentom
- 4.5 Štruktúra konfigurácie a spúšťania experimentov

Kapitola 5

Experimentálne overenie

- 5.1 Výber testovaných projektov
- 5.2 Experiment s knižnicou validators
- 5.3 Porovnanie základného a cieleného spustenia
- 5.4 Vyhodnotenie času behu a počtu mutantov
- 5.5 Analýza preživších mutantov

Kapitola 6

Diskusia

6.1 Prínosy navrhnutého riešenia

6.2 Obmedzenia prototypu

6.3 Možnosti využitia veľkých jazykových modelov

6.4 Hrozby pre platnosť vyhodnotenia

Kapitola 7

Záver

7.1 Zhrnutie výsledkov

7.2 Splnenie cieľov práce

7.3 Možnosti ďalšieho rozšírenia

Literatúra

- [1] Mark Harman, Jillian Ritchey, Inna Harper, Shubho Sengupta, Ke Mao, Abhishek Gulati, Christopher Foster, and Hervé Robert. Mutation-guided llm-based test generation at meta. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, FSE Companion '25, page 180–191, New York, NY, USA, 2025. Association for Computing Machinery.
- [2] Ana B. Sánchez, José Parejo, Sergio Segura, Amador Durán Toro, and Mike Papadakis. Mutation testing in practice: Insights from open-source software developers. *IEEE Transactions on Software Engineering*, PP:1–15, 05 2024.